Interactive Proofmode for Temporal Logic in Rocq

Jonas Kastberg Hinrichsen
Assistant Professor
Aalborg University

Elli Anastasiadi
Assistant Professor
Aalborg University

20. August, CoPLaWS 2026

Slides:

Context:

We care about liveness properties and mechanisation

Context:

We care about liveness properties and mechanisation

Problem:

Limited support for mechanising temporal properties

Context:

We care about liveness properties and mechanisation

Problem:

Limited support for mechanising temporal properties

Solution:

Interactive Proofmode for Temporal Logic in Rocq

Context:

We care about liveness and mechanisation

Liveness Properties and Temporal Logic

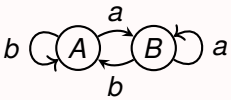
Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces



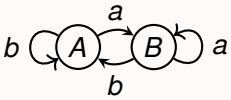
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



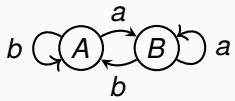
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...
b,b,b,...
b..., a..., b...

Liveness Properties and Temporal Logic

Liveness: something good eventually happens

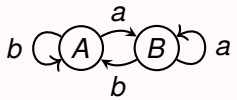
- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\mathsf{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\mathsf{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\mathsf{tail}^n\, tr)$



a,b,a,...
b,b,b,...
b..., a..., b...

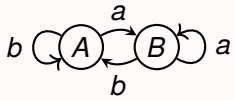
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...
b,b,b,...
b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\mathsf{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\mathsf{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\mathsf{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$

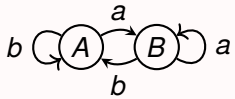
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \text{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...
b,b,b,...
b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\text{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\text{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\text{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$
- Axioms: $A \wedge a \vdash \circ B$ $\quad A \wedge b \vdash \circ A$

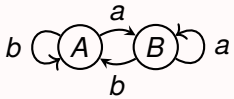
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...
b,b,b,...
b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\mathsf{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\mathsf{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\mathsf{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$
- Axioms: $A \wedge a \vdash \circ B$ $\quad A \wedge b \vdash \circ A$
- Fairness assumption: $\Box \Diamond a$

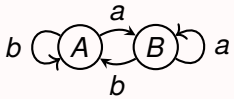
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \mathsf{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...

~~b,b,b,...~~

b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\mathsf{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\mathsf{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\mathsf{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$
- Axioms: $A \wedge a \vdash \circ B$ $\quad A \wedge b \vdash \circ A$
- Fairness assumption: $\Box \Diamond a$

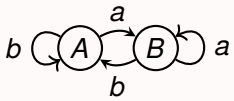
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \text{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...

~~b,b,b,...~~

b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\text{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\text{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\text{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$
- Axioms: $A \wedge a \vdash \circ B \qquad A \wedge b \vdash \circ A$
- Fairness assumption: $\Box \Diamond a$
- One derivation step: $\Box(A \to \circ \Diamond B) \vdash \Diamond A \to \circ \Diamond B$

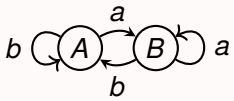
Liveness Properties and Temporal Logic

Liveness: something good eventually happens

- E.g.: Collaborating threads eventually make progress under fair scheduling
- Often studied in terms of labelled transition system (LTS) traces

Linear Temporal logic: logic for reasoning about traces

- $P \vdash Q \triangleq \forall tr.\, P\, tr \to Q\, tr$ where $P, Q \in \texttt{Trace} \to \text{Prop}$
- Here, `Trace` is a maximal labelled trace w.r.t. some LTS



a,b,a,...

~~b,b,b,...~~

b..., a..., b...

Most temporal logics are based on modalities, such as

- Next P: $\circ P \equiv \lambda tr.\, P(\text{tail}\, tr)$
- Globally P: $\Box P \equiv \lambda tr.\, \forall n.\, P(\text{tail}^n\, tr)$
- Eventually P: $\Diamond P \equiv \lambda tr.\, \exists n.\, P(\text{tail}^n\, tr)$

Example:

- Property: $\Diamond A \vdash \circ \Diamond B$
- Axioms: $A \wedge a \vdash \circ B \qquad A \wedge b \vdash \circ A$
- Fairness assumption: $\Box \Diamond a$
- One derivation step: $\Box(P \to \circ \Diamond Q) \vdash \Diamond P \to \circ \Diamond Q$

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction

```
From Stdlib Require Import Lia.

Fixpoint fac n :=
  match n with
  | 0 ⇒ 1
  | S n' ⇒ n * fac n'
  end.

Compute fac 5.

Lemma fac_succ n : fac (S n) = S n * fac n.
Proof. reflexivity. Qed.

Lemma fac_pos n : fac n > 0.
Proof.
 induction n.
 - simpl. lia.
 - rewrite fac_succ. lia.
Qed.
```

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction
- Time consuming

```
From Stdlib Require Import Lia.

Fixpoint fac n :=
  match n with
  | 0 ⇒ 1
  | S n' ⇒ n * fac n'
  end.

Compute fac 5.

Lemma fac_succ n : fac (S n) = S n * fac n.
Proof. reflexivity. Qed.

Lemma fac_pos n : fac n > 0.
Proof.
  induction n.
  - simpl. lia.
  - rewrite fac_succ. lia.
Qed.
```

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction
- Time consuming, especially with limited support

```
Definition is_adjoint' {A B : Type} (f : A → B) (g : B → A)
           (issect : g∘ f = id) (isretr : f ∘ g = id) (a : A) :
  isretr (f a) = ap f (issect' f g issect isretr a).⋄
Proof.⋄
  unfold issect'.⋄
  apply moveL_M1.⋄
  repeat rewrite ap_pp.⋄ rewrite ← concat_p_pp; rewrite ← ap_compose.⋄
  pose  (concat_pA1 (fun b ⇒ (isretr b)^) (ap f (issect a))).⋄
  eapply concat.⋄ 2: {⋄ apply ap2.⋄ reflexivity.⋄ exact e. }⋄
  cbn.⋄ clear e.⋄
  pose (concat_pA1 (fun b ⇒ (isretr b)^) (isretr (f a))).⋄
  rewrite ← concat_p_pp.⋄
  pose (concat_pA1p (fun b ⇒ (isretr b)) (isretr (f a))).⋄
  apply moveL_Vp in e0.⋄
  rewrite ← concat_p_pp in e0.⋄ rewrite inv_inv' in e0.⋄
  rewrite concat_refl in e0.⋄
  rewrite ap_compose in e0.⋄
  eapply concat.⋄
  2: {⋄ apply ap2.⋄ reflexivity.⋄ rewrite concat_p_pp.⋄ eapply
concat.⋄
      2: {⋄ apply ap2.⋄ eapply concat.⋄
           2:{⋄ apply ap2.⋄ symmetry.⋄ apply e0.⋄ reflexivity. }⋄
             symmetry.⋄ apply inv_inv'.⋄ reflexivity. }⋄
           reflexivity. }⋄
  repeat rewrite ← ap_compose.⋄
  cbn.⋄ symmetry.⋄ eapply concat.⋄ refine (ap_pp ((f ∘ g) ∗f) _ _)^.⋄
  rewrite inv_inv.⋄ reflexivity.
```

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction
- Time consuming, especially with limited support
- Support: libraries, tactics, verified SMT solvers, ...

```
Definition is_adjoint' {A B : Type} (f : A → B) (g : B → A)
           (issect : g∘ f == id) (isretr : f  ∘ g == id) (a : A) :
  isretr (f a) = ap f (issect' f g issect isretr a).
Proof.
  unfold issect'.
  apply moveL_M1.
  repeat rewrite ap_pp. rewrite ← concat_p_pp; rewrite ← ap_compose.
  pose  (concat_pA1 (fun b ⇒ (isretr b)^) (ap f (issect a))).
  eapply concat. 2: { apply ap2. reflexivity. exact e. }
  cbn. clear e.
  pose (concat_pA1 (fun b ⇒ (isretr b)^) (isretr (f a))).
  rewrite ← concat_p_pp.
  pose (concat_A1p (fun b ⇒ (isretr b)) (isretr (f a))).
  apply moveL_Vp in e0.
  rewrite ← concat_p_pp in e0. rewrite inv_inv' in e0.
  rewrite concat_refl in e0.
  rewrite ap_compose in e0.
  eapply concat.
  2: { apply ap2. reflexivity. rewrite concat_p_pp. eapply
concat.
       2: { apply ap2. eapply concat.
            2:{ apply ap2. symmetry. apply e0. reflexivity. }
            symmetry. apply inv_inv'. reflexivity. }
  repeat rewrite ← ap_compose.
  cbn. symmetry. eapply concat. refine (ap_pp ((f ∘ g) ∘f) _ _)^.
  rewrite inv_inv. reflexivity.
```

Mechanisation

Mechanisation: machine-checked proofs

- Trust through rigor; no stone unturned
- Small trusted computing base
- Reusable proof infrastructure
- Increasingly becoming a community expectation
  - "All results have been mechanised"

Interactive theorem provers (ITPs): Rocq, Lean, ...

- Interactive step-wise deduction
- Time consuming, especially with limited support
- Support: libraries, tactics, verified SMT solvers, ...

```
Local Lemma try_acquire_spec γ lk R :
  {{{ is_lock γ lk R }}} try_acquire lk
  {{{ b, RET #b; if b is true then locked γ ∗ R else True }}}.
Proof.
  iIntros (Φ) "#Hl HΦ". iDestruct "Hl" as (l ->) "#Hinv".
  wp_rec. wp_bind (CmpXchg _ _ _). iInv N as ([]) "[Hl HR]".
  - wp_cmpxchg_fail
    wp_pures. iApp
  - wp_cmpxchg_suc
    iModIntro. iSp
    rewrite /locke
Qed.

Local Lemma acquir
  {{{ is_lock γ lk
Proof.
  iIntros (Φ) "#Hl
  wp_apply (try_ac
  - iIntros "[Hlked HR]". wp_if. iApply "HΦ"; auto with iFrame.
  - iIntros "_". wp_if. iApply ("IH" with "[HΦ]"). auto.
Qed.
```

Problem:

Limited support for mechanising temporal properties

(in Rocq)

Mechanising Temporal Properties in Rocq

Option 1: Going nuclear

- Give a paper proof / (unverified) translation to model checker
- Increases the trusted computing base
- Remove “All results have been mechanised” statement

Mechanising Temporal Properties in Rocq

Option 1: Going nuclear

- Give a paper proof / (unverified) translation to model checker
- Increases the trusted computing base
- Remove “All results have been mechanised” statement

Option 2: Do it yourself

- Mechanise proof ad hoc
- Given a custom trace definition with ever-growing suite of lemmas

Mechanising Temporal Properties in Rocq

Option 1: Going nuclear

- Give a paper proof / (unverified) translation to model checker
- Increases the trusted computing base
- Remove “All results have been mechanised” statement

Option 2: Do it yourself

- Mechanise proof ad hoc
- Given a custom trace definition with ever-growing suite of lemmas

Option 3: Use existing libraries

- Browse selection of existing (LTL) libraries in Rocq
- They focus on metatheory => limited helper lemmas / tactic support

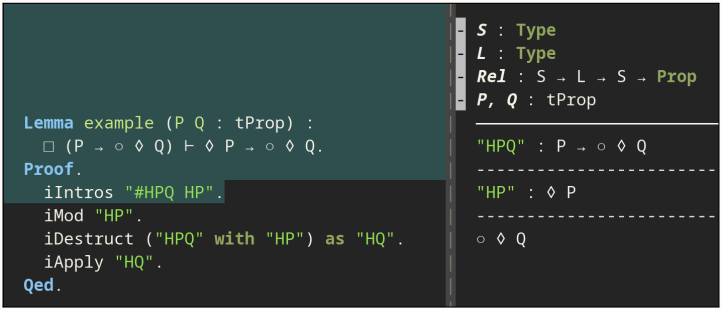
Solution:

Interactive Proofmode for Temporal Logic in Rocq

Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

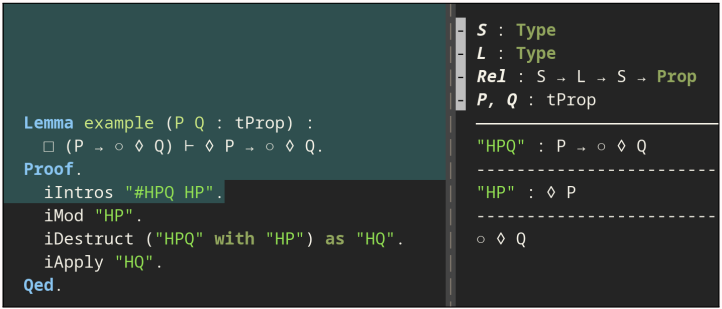
- Proofmode context for temporal logic



Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

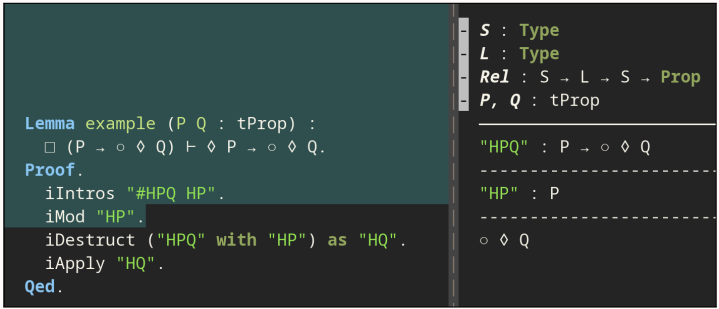
- Proofmode context for temporal logic
- Tactics for working with temporal modalities



Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

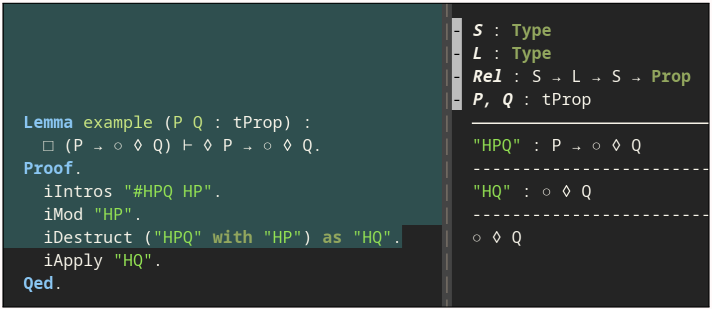
- Proofmode context for temporal logic
- Tactics for working with temporal modalities



Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

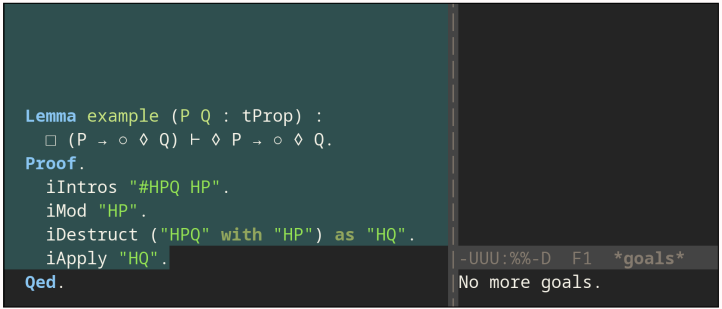
- Proofmode context for temporal logic
- Tactics for working with temporal modalities



Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

- Proofmode context for temporal logic
- Tactics for working with temporal modalities



Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

- Proofmode context for temporal logic
- Tactics for working with temporal modalities

LTL for maximal possibly-finite traces

- Primitive Next ($\circ$) modality (even for terminated traces)
- Derived Globally ($\Box$) modality via Next

Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

- Proofmode context for temporal logic
- Tactics for working with temporal modalities

LTL for maximal possibly-finite traces

- Primitive Next ($\circ$) modality (even for terminated traces)
- Derived Globally ($\Box$) modality via Next

Support for least and greatest fixpoints

- Logic-level fixpoints with fixpoint theorems for free
- E.g. $\Diamond P \triangleq \mu X.\, P \vee \circ X$ $\quad$ $\mathsf{every\_other}\, P \triangleq \nu X.\, P \wedge \circ \circ P$

Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

- Proofmode context for temporal logic
- Tactics for working with temporal modalities

LTL for maximal possibly-finite traces

- Primitive Next ($\circ$) modality (even for terminated traces)
- Derived Globally ($\Box$) modality via Next

Support for least and greatest fixpoints

- Logic-level fixpoints with fixpoint theorems for free
- E.g. $\Diamond P \triangleq \mu X.\, P \vee \circ X$ $\quad$ $\mathsf{every\_other}\, P \triangleq \nu X.\, P \wedge \circ \circ P$

Infrastructure for instantiation with custom LTS models

- Reflect LTS stepping relation as logic-level axioms
- Extract meta-level theorems about traces

Contributions

Introducing Modal Temporal Logic Proofmode (MoTeL)

- Proofmode context for temporal logic
- Tactics for working with temporal modalities

LTL for maximal possibly-finite traces

- Primitive Next ($\circ$) modality (even for terminated traces)
- Derived Globally ($\Box$) modality via Next

Support for least and greatest fixpoints

- Logic-level fixpoints with fixpoint theorems for free
- E.g. $\Diamond P \triangleq \mu X.\, P \vee \circ X$ $\quad$ $\mathsf{every\_other}\, P \triangleq \nu X.\, P \wedge \circ \circ P$

Infrastructure for instantiation with custom LTS models

- Reflect LTS stepping relation as logic-level axioms
- Extract meta-level theorems about traces

Proof of eventual delivery for model of Stenning protocol

- In an unreliable network with a fair delivery assumption

Modal Temporal Logic Proofmode Demo

Developing the Modal Temporal Logic Proofmode

MoSeL: Modal Separation Logic (Proofmode)

- "Extensible Modal Framework for Interactive Proofs (in Separation Logic)"
- Foundation for the Iris Proof Mode (facilitating 150+ papers since 2015)
- Now also foundation for the Modal Temporal Logic Proofmode

Developing the Modal Temporal Logic Proofmode

MoSeL: Modal Separation Logic (Proofmode)

- "Extensible Modal Framework for Interactive Proofs (in Separation Logic)"
- Foundation for the Iris Proof Mode (facilitating 150+ papers since 2015)
- Now also foundation for the Modal Temporal Logic Proofmode

Key features of MoSeL

- Managed proof context for spatial and persistent propositions
- Extensible handling of modality-based reasoning via typeclasses
- Least and greatest monotone fixpoint constructions
- (Opt-in) step-indexing and separation logic
- Coming to Lean!

Developing the Modal Temporal Logic Proofmode

MoSeL: Modal Separation Logic (Proofmode)

- "Extensible Modal Framework for Interactive Proofs (in Separation Logic)"
- Foundation for the Iris Proof Mode (facilitating 150+ papers since 2015)
- Now also foundation for the Modal Temporal Logic Proofmode

Key features of MoSeL

- Managed proof context for spatial and persistent propositions
- Extensible handling of modality-based reasoning via typeclasses
- Least and greatest monotone fixpoint constructions
- (Opt-in) step-indexing and separation logic
- Coming to Lean!

Key insights

- Modality laws of Iris modalities resemble LTL modalities
- Separation logic and other unused features do not interfere

To Conclude

Future Work

More case studies

- Looking for input and collaboration opportunities

Future Work

More case studies

- Looking for input and collaboration opportunities

Temporal logic beyond linear

- Computation tree logic (CTL)
- Branching time μ-calculus $\rightarrow$ derived LTL and CTL

Future Work

More case studies

- Looking for input and collaboration opportunities

Temporal logic beyond linear

- Computation tree logic (CTL)
- Branching time μ-calculus $\rightarrow$ derived LTL and CTL

Temporal separation logic

- Conceptually stimulating
- Practically unclear what it even means
- MoTeL might make it more feasible to test the waters

Conclusion

Support for mechanisation of temporal properties is warranted

- Currently limited support
- ITP-level solution preserves small TCB

Conclusion

Support for mechanisation of temporal properties is warranted

- Currently limited support
- ITP-level solution preserves small TCB

Modal temporal logic is suitable for interactive verification

- Modal laws facilitate smooth proof flows

Conclusion

Support for mechanisation of temporal properties is warranted

- Currently limited support
- ITP-level solution preserves small TCB

Modal temporal logic is suitable for interactive verification

- Modal laws facilitate smooth proof flows

MoSeL seems mature enough to handle new domain logics

- Relatively easy to instantiate
- Framework can handle functionality beyond original intent
- I encourage trying it out!

$$\vdash \Box(\mathit{Question} \rightarrow \circ \Diamond \mathit{Answer})$$

Slides available at:

Rocq code available at: